hey everyone welcome back to the channel I hope you guys are doing extremely well so...

Revision Notes (Striver A2Z DSA – Arrays): Rotation, Move Zeros, Linear Search, Union & Intersection

---

1) Left Rotate an Array by **1** Place

Idea (In-place, optimal)

- Save first element.
- Shift all elements left by 1.
- Put saved element at end.

Steps

1. `temp = a[0]`
2. For `i = 1 .. n-1`: set `a[i-1] = a[i]`
3. `a[n-1] = temp`

Pseudocode

```
temp = a[0]
for i = 1 to n-1:
   a[i-1] = a[i]
a[n-1] = temp
```

Complexity

- Time: $O(n)$
- Extra space: $O(1)$

Interview note: clarify “extra space” vs “space used by input array”. Extra space is $O(1)$; the input array exists anyway.

---

2) Left Rotate an Array by **D** Places

Key observation

Rotating by array size returns to original:

- Effective rotations: `d = d % n`

Example: `n = 7, d = 15` ⟶ `d = 15 % 7 = 1`

A) Brute Force (uses extra temp of size D)

Steps

1. `d = d % n`
2. Store first `d` elements in temp.
3. Shift remaining elements left by `d`.
4. Copy temp to last `d` positions.

Pseudocode

```
d = d % n
temp = new array of size d
for i = 0 to d-1:
   temp[i] = a[i]

for i = d to n-1:
   a[i-d] = a[i]

for i = n-d to n-1:
   a[i] = temp[i-(n-d)]   // or use a j pointer from 0
```

Complexity

- Time: $O(n)$
- Extra space: $O(d)$

B) Optimal (3-Reversal Method, in-place)

Observation

For left rotate by `d`:

- Final array = `[a[d..n-1]] + [a[0..d-1]]`
  Using reversals:

1. Reverse first `d` elements
2. Reverse remaining `n-d` elements
3. Reverse whole array

Pseudocode

```
d = d % n
reverse(a, 0, d-1)
reverse(a, d, n-1)
reverse(a, 0, n-1)
```

Complexity

- Time: $O(n)$
- Extra space: $O(1)$

Assignment hint (Right rotation by D):

- Right rotate by `d` = left rotate by `n-d`
- Or reversal version:

```
d = d % n
reverse(a, 0, n-d-1)
reverse(a, n-d, n-1)
reverse(a, 0, n-1)
```

---

3) Move All Zeros to the End

Goal: move all `0`s to end while keeping relative order of non-zeros (typical expectation).

A) Brute Force (temp list of non-zeros)

Steps

1. Collect all non-zero elements into temp.

2. Write them back to array from start.
3. Fill remaining positions with 0.

Complexity

- Time: $O(n)$
- Extra space: $O(x)$ where x = number of non-zeros (worst $O(n)$)

B) Optimal (Two-Pointer In-place)

Core idea

- Find first zero index `j`.
- Scan from `j+1` with `i`. When `a[i] != 0`, swap `a[i]` and `a[j]`, then `j++`.

Pseudocode

```
j = -1
for i = 0 to n-1:
   if a[i] == 0:
      j = i
      break
if j == -1: return  // no zeros

for i = j+1 to n-1:
   if a[i] != 0:
      swap(a[i], a[j])
      j++
```

Complexity

- Time: $O(n)$
- Extra space: $O(1)$

---

4) Linear Search (First Occurrence)

Problem

Given array `a` and value `x`, return index of **first occurrence**, else `-1`.

Pseudocode

```
for i = 0 to n-1:
    if a[i] == x:
        return i
return -1
```

Complexity

- Time: $O(n)$
- Extra space: $O(1)$

---

5) Union of Two **Sorted** Arrays (unique, sorted result)

A) Brute Force using Set

- Insert all elements from both arrays into an ordered set.
- Output elements from set.

Complexity

- Time: ~ $O((n1 + n2)\log(n1 + n2))$
- Extra space: $O(n1 + n2)$ (worst case all unique)

B) Optimal Two-Pointer (since arrays are sorted)

Idea

Walk both arrays; always pick smaller element, and avoid duplicates by checking last inserted.

Pseudocode

```
i = 0, j = 0
union = empty list

while i < n1 and j < n2:
```

```
    if a[i] <= b[j]:
        if union empty OR union.last != a[i]:
            union.push(a[i])
        i++
    else:
        if union empty OR union.last != b[j]:
            union.push(b[j])
        j++

while i < n1:
    if union empty OR union.last != a[i]:
        union.push(a[i])
    i++

while j < n2:
    if union empty OR union.last != b[j]:
        union.push(b[j])
    j++
```

Complexity

- Time: $O(n1 + n2)$
- Extra space: $O(n1 + n2)$ for output (not “extra working memory”)

---

6) Intersection of Two **Sorted** Arrays

Intersection = elements present in both; duplicates appear as many times as they match.

A) Brute Force (visited array)

- For each element in A, scan B to find an unused matching element.
- Mark that B index as used.

Complexity

- Time: $O(n1 \cdot n2)$
- Extra space: $O(n2)$ (or smaller-array size)

B) Optimal Two-Pointer

Idea

- If `a[i] < b[j]` → `i++` (a[i] can't match later)
- If `a[i] > b[j]` → `j++`
- If equal → add to answer, `i++`, `j++`

Pseudocode

```
i = 0, j = 0
inter = empty list

while i < n1 and j < n2:
   if a[i] < b[j]:
      i++
```